

Continuing with the plugin review for "artherslin". Let's dive in!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please **check this email thoroughly**, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

The URL(s) declared in your plugin seems to be invalid or does not work.

From your plugin:

Author URI: <https://museder.com/> – museder-restoreone.php – TLS connect error: error:0A000126:SSL routines::unexpected eof while reading

Plugin URI: <https://museder.com/restoreone> – museder-restoreone.php – TLS connect error: error:0A000126:SSL routines::unexpected eof while reading

Undocumented use of a 3rd Party / external service

Plugins are permitted to require the **use of third party/external services** as long as they are **clearly documented**.

When your plugin reach out to external services, you must disclose it. This is true even if you are the one providing that service.

You are required to document it in a clear and plain language, so users are aware of: **what data is sent, why, where and under which conditions**.

To do this, you must update your readme file to **clearly explain that your plugin relies on third party/external services**, and include **at least the following information** for each third party/external service that this plugin uses:

- What the service is and what it is used for.
- What data is sent and when.
- Provide links to the service's terms of service and privacy policy.

Remember, this is for your own legal protection. Use of services must be upfront and well documented. This allows users to ensure that any legal issues with data transmissions are covered.

Example:

== External services ==

This plugin connects to an API to obtain weather information, it's needed to show the weather information and forecasts in the included widget.

It sends the user's location every time the widget is loaded (If the location isn't available and/or the user hasn't

given their consent, it displays a configurable default location).

This service is provided by "PRT Weather INC": [terms of use](#), [privacy policy](#).

Example(s) from your plugin:

Domain(s) not mentioned in the readme file.

```
includes/class-backup-lite-s3-service.php:1192 $host = 's3.' . $region . '.amazonaws.com';
includes/class-cloud-service.php:211 : sprintf( 'https://s3.%s.amazonaws.com', $settings['s3_region'] );
includes/class-backup-lite-s3-service.php:1190 $host = $bucket . 's3.' . $region . '.amazonaws.com';
```

Determine files and directories locations correctly

WordPress provides several functions for easily determining where a given file or directory lives.

We detected that **the way your plugin references some files**, directories and/or URLs **may not work with all WordPress setups**. This happens because there are hardcoded references or you are using the WordPress internal constants.

Let's improve it, please check out the following documentation:

<https://developer.wordpress.org/plugins/plugin-basics/determining-plugin-and-content-directories/>

It contains all the functions available to determine locations correctly.

Most common cases in plugins can be solved using the following functions:

- For where your plugin is located: `plugin_dir_path()` , `plugin_dir_url()` , `plugins_url()`
- For the uploads directory: `wp_upload_dir()` (*Note: If you need to write files, please do so in a folder in the uploads directory, not in your plugin directories*).

Example(s) from your plugin:

```
includes/class-restore.php:620 'mu-plugins' => WP_CONTENT_DIR . '/mu-plugins',
includes/class-restore.php:618 'plugins'    => WP_CONTENT_DIR . '/plugins',
includes/class-backup.php:1033 WP_CONTENT_DIR . '/uploads/backup-lite/jobs',
includes/class-restore-service.php:782 self::recursive_copy( $content_dir, WP_CONTENT_DIR );
includes/class-backup.php:1037 WP_CONTENT_DIR . '/uploads/backup-lite/logs',
includes/class-restore-handler.php:1963 $plugins_dir = WP_PLUGIN_DIR;
includes/class-backup.php:407 'uploads' => WP_CONTENT_DIR . '/uploads',
includes/class-backup.php:1035 WP_CONTENT_DIR . '/uploads/backup-lite/pro/jobs',
```

... out of a total of 30 incidences.

 If your plugin creates files, those should be saved in the uploads directory.

In order to determine that directory location you can make use of the `wp_upload_dir()` function.

Also, it's a good practice that you store those files under a folder with the same name as your plugin slug "museder-restoreone"

Example:

```
$path = wp_upload_dir()['basedir'] . '/museder-restoreone';
```

Example(s) from your plugin:

```
includes/class-backup.php:1031 WP_CONTENT_DIR . '/uploads/backup-lite/backups',
includes/class-backup.php:1033 WP_CONTENT_DIR . '/uploads/backup-lite/jobs',
```

... out of a total of 11 incidences.

i In order to determine your plugin location, you would need to use the `__FILE__` variable for this to work properly. *Note that this variable depends on the location of the file making the call. As this can create confusion, a common practice is to save its value in a `define()` in the main file of your plugin so that you don't have to worry about this.*

Example: Your main plugin file.

```
define( 'MUSERE_PLUGIN_FILE', __FILE__ );
define( 'MUSERE_PLUGIN_DIR', plugin_dir_path( __FILE__ ) );
define( 'MUSERE_PLUGIN_URL', plugin_dir_url( __FILE__ ) );
```

Example: Any file of your plugin.

```
require_once MUSERE_PLUGIN_DIR . 'admin/class-init.php';
```

```
function musere_scripts() {
    wp_enqueue_script( 'musere-script', MUSERE_PLUGIN_URL . 'js/script.js', array(), MUSERE_VERSION );
    // Or alternatively
    wp_enqueue_script( 'musere-script', plugins_url( 'js/script.js', MUSERE_PLUGIN_FILE ), array(),
MUSERE_VERSION );
}
add_action( 'wp_enqueue_scripts', 'musere_scripts' );
```

Example(s) from your plugin:

```
includes/class-backup.php:406 'plugins' => WP_CONTENT_DIR . '/plugins',
includes/class-restore-handler.php:1963 $plugins_dir = WP_PLUGIN_DIR;
```

... out of a total of 7 incidences.

Don't Use Error Reporting in Production Code

While `error_reporting()` is a great tool in PHP (<https://www.php.net/manual/en/function.error-reporting.php>) but if you set it permanently in your plugin, you mess things up for everyone who uses your code. Should they have a reason to try to debug their site which happens to use your code, they won't be able to get a clean test because you're messing with the output. It has no place in the day to day function of your plugin.

Example(s) from your plugin:

includes/class-chunk-handler-v2.php:18 error_reporting(E_ALL & ~E_NOTICE & ~E_WARNING);

Using CURL Instead of HTTP API

WordPress comes with an extensive HTTP API that should be used instead of creating your own curl calls. It's both faster and more extensive. It'll fall back to curl if it has to, but it'll use a lot of WordPress' native functionality first.

<https://developer.wordpress.org/plugins/http-api/>

In case you need, you can use setopt with https://developer.wordpress.org/reference/hooks/http_api_curl/

Please note: If you're using CURL in 3rd party vendor libraries, that's permitted. It's in your own code unique to this plugin (or any dedicated WordPress libraries) that we need it corrected.

Example(s) from your plugin:

includes/class-backup-lite-s3-service.php:858 curl_exec(\$ch);

Changing Active Plugins

It is not allowed for plugins to change the activation status of other plugins, this is an action that must be performed by the user.

It is also not allowed to interfere with the user's actions when activating or deactivating a plugin, unless that's done to prevent errors (For example: When your plugin depends on another plugin, deactivate your own plugin when that other plugin is not active).

WordPress 6.5 introduces [Plugin Dependencies](#), you can use it to manage dependencies (although it's fine if you use this as a fallback).

From your plugin:

includes/class-restore-service.php:1207 update_option('active_plugins', \$valid_active_plugins);
includes/class-restore-handler.php:2004 update_option('active_plugins', \$restored_order);
includes/class-restore.php:1955 update_option('active_plugins', \$valid_plugins, false);
includes/class-restore.php:1897 update_option('active_plugins', \$essential_plugins, false);

Nonces and User Permissions Needed for Security

Please add a nonce check to your input calls (\$_POST, \$_GET, \$REQUEST) to prevent unauthorized access.

If you use wp_ajax_ to trigger submission checks, remember they also need a nonce check.

 Checking permissions: Keep in mind, a nonce check alone is not bulletproof security. Do not rely on nonces for authorization purposes. When needed, use it together with current_user_can() in order to prevent users without the right permissions from accessing things they shouldn't.

Also make sure that the **nonce logic is correct by making sure it cannot be bypassed**. Checking the nonce with `current_user_can()` is great, but mixing it with other checks can make the condition more complex and, without realising it, bypassable, remember that **anything** can be sent through an input, **don't trust any input**.

Keep performance in mind. Don't check for post submission outside of functions. Doing so means that the check will run on every single load of the plugin, which means that every single person who views any page on a site using your plugin will be checking for a submission. This will make your code slow and unwieldy for users on any high traffic site, leading to instability and eventually crashes.

The following links may assist you in development:

- <https://developer.wordpress.org/plugins/security/nonces/>
- <https://developer.wordpress.org/plugins/javascript/ajax/#nonce>
- <https://developer.wordpress.org/plugins/settings/settings-api/>

From your plugin:

```
includes/class-ui.php:395 Backup_Lite_UI::ajax_cancel_backup_job() [classMethod] No nonce check found
validating input origin on lines 395-405
# ↳ Line 399: $job_id = isset( $_POST['job_id'] ) ? sanitize_text_field( wp_unslash( $_POST['job_id'] ) ) : '';
includes/class-ui.php:414 Backup_Lite_UI::handle_restore_request() [classMethod] No nonce check found validating
input origin on lines 414-430
# ↳ Line 430: if ( isset( $_FILES['restoreFile'] ) && is_uploaded_file( $_FILES['restoreFile']['tmp_name'] ) ) {
# ↳ Line 431: $uploaded_file = $_FILES['restoreFile'];
# ↳ Line 432: } elseif ( isset( $_FILES['restore_file'] ) && is_uploaded_file( $_FILES['restore_file']['tmp_name'] ) ) {
# ↳ Line 433: $uploaded_file = $_FILES['restore_file'];
includes/pro/reports-controller.php:182 Backup_Lite_Reports_Controller::handle_report_download() [classMethod]
No nonce check found validating input origin on lines 182-197
# ↳ Line 189: $file = sanitize_text_field( wp_unslash( $_GET['file'] ?? '' ) );
includes/class-ui.php:918 Backup_Lite_UI::get_backup_options_from_request() [classMethod] No nonce check
found validating input origin on lines 918-929
# ↳ Line 925: $backup_label = sanitize_text_field( wp_unslash( $_POST['backup_label'] ) );
# ↳ Line 943: $backup_cloud_raw = wp_unslash( $_POST['backup_cloud'] );
includes/class-log-handler.php:175 Backup_Lite_Log_Handler::ajax_download_log() [classMethod] No nonce check
found validating input origin on lines 175-182
# ↳ Line 180: $log = isset( $_GET['log'] ) ? sanitize_text_field( wp_unslash( $_GET['log'] ) ) : '';
```

... out of a total of 11 incidences.

Data Must be Sanitized, Escaped, and Validated

When you include POST/GET/REQUEST/FILE calls in your plugin, it's important to sanitize, validate, and escape them. The goal here is to prevent a user from accidentally sending trash data through the system, as well as protecting them from potential security issues.

SANITIZE: Data that is input (either by a user or automatically) must be sanitized as soon as possible. This lessens the possibility of XSS vulnerabilities and MITM attacks where posted data is subverted.

VALIDATE: All data should be validated, no matter what. Even when you sanitize, remember that you don't want someone putting in 'dog' when the only valid values are numbers.

ESCAPE: Data that is output must be escaped properly when it is echo'd, so it can't hijack admin screens. There

are many `esc_*()` functions you can use to make sure you don't show people the wrong data.

To help you with this, WordPress comes with a number of sanitization and escaping functions. You can read about those here:

- <https://developer.wordpress.org/apis/security/sanitizing/>
- <https://developer.wordpress.org/apis/security/escaping/>

Remember: You must use the most appropriate functions for the context. If you're sanitizing email, use `sanitize_email()`, if you're outputting HTML, use `wp_kses_post()`, and so on.

An easy mantra here is this:

Sanitize *early*
Escape *Late*
Always Validate

Clean everything, check everything, escape everything, and never trust the users to always have input sane data. After all, users come from all walks of life.

Example(s) from your plugin:

```
includes/class-restore-handler.php:1277 $chunk_file = isset( $_FILES['chunk'] ) && is_array( $_FILES['chunk'] ) ?
$_FILES['chunk'] : null;
# ↳ Line 1283: if ( ! is_array( $chunk_file ) || UPLOAD_ERR_OK !== $chunk_err || $chunk_size <= 0 || empty(
$chunk_tmp ) || ! is_uploaded_file( $chunk_tmp ) || ! file_exists( $chunk_tmp ) ) {
# ↳ Line 1283: if ( ! is_array( $chunk_file ) || UPLOAD_ERR_OK !== $chunk_err || $chunk_size <= 0 || empty(
$chunk_tmp ) || ! is_uploaded_file( $chunk_tmp ) || ! file_exists( $chunk_tmp ) ) {
# ↳ Line 1303: $input = fopen( $chunk_tmp, 'rb' );
includes/class-ui.php:616 $search_replace_raw = wp_unslash( $_POST['search_replace'] );
# ↳ Line 624: $options['search_replace'] = array_map( function( $item ) {
if ( is_array( $item ) ) {
return array_map( 'sanitize_text_field', $item );
}
return sanitize_text_field( $item );
}, $decoded );
includes/class-chunk-handler.php:178 $candidates[] = $_FILES['chunk'];
# ↳ Line 184: if ( UPLOAD_ERR_OK === $err && $size > 0 && ! empty( $tmp ) && is_uploaded_file( $tmp ) &&
file_exists( $tmp ) ) {
# ↳ Line 184: if ( UPLOAD_ERR_OK === $err && $size > 0 && ! empty( $tmp ) && is_uploaded_file( $tmp ) &&
file_exists( $tmp ) ) {
# ↳ Line 199: $input = fopen( $uploaded_file, 'rb' );
includes/class-restore-handler.php:59 $file = $_FILES['restore_file'];
# ↳ Line 198: require_once plugin_dir_path( __FILE__ ) . 'class-ai1wm-converter.php';
```

Note: While the `json_decode()` function in PHP is useful for decoding JSON strings, it does not sanitize the input. Sanitization refers to the process of cleaning or filtering the input data to ensure that it is safe and secure to use.

The `json_decode()` function simply transforms a JSON string into a PHP array or object. Any potentially malicious data or scripts may persist after `json_decode()`.

Example(s) from your plugin:


```
includes/class-ui.php:621 $decoded = json_decode( $search_replace_raw, true );
includes/class-ui.php:951 $decoded = json_decode( $cloud, true );
includes/class-ui.php:725 $decoded = json_decode( wp_unslash( $raw ), true );
includes/class-settings.php:194 $decoded = json_decode( $data, true );
```

... out of a total of 5 incidences.

✓ You can check this using [Plugin Check](#).

Processing the whole input

We strongly recommend you **never** attempt to process the whole \$_POST/\$_REQUEST/\$_GET stack. This makes your plugin slower as you're needlessly cycling through data you don't need. Instead, you should only be attempting to process the items within that are required for your plugin to function.

Example(s) from your plugin:

```
includes/class-schedule-handler.php:796 $raw = file_get_contents( 'php://input' ); // Fallback for JSON payload.
includes/class-schedule-handler.php:805 foreach ( $_POST as $key => $value ) {
if ( 'schedule' !== $key && 'nonce' !== $key ) {
if ( is_array( $value ) ) {
$decoded[ sanitize_key( $key ) ] = array_map( 'sanitize_text_field', array_map( 'wp_unslash', $value ) );
} else {
$decoded[ sanitize_key( $key ) ] = sanitize_text_field( wp_unslash( $value ) );
}
}
}
includes/class-settings.php:193 $data = file_get_contents( 'php://input' );
```

Generic function/class/define/namespace/option names

All plugins must have unique function names, namespaces, defines, class and option names. This prevents your plugin from conflicting with other plugins or themes. We need you to update your plugin to use more unique and distinct names.

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', \$options);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global \$musere_options;
- add_action('wp_ajax_musere_save_data', ...);
- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers.

Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if (!function_exists('NAME')) {` around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using `if-exists` should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

This plugin is using the prefix "backup_lite" for 107 element(s).

Looks like there are elements not using common prefixes.

```
includes/class-ui.php:97 wp_localize_script('backup-lite-admin-ui', 'BackupLitePro', ['isPro' => $is_pro,
'upgradeUrl' => class_exists('Backup_Lite_Pro') ? Backup_Lite_Pro::get_upgrade_url() : 'https://your-site.com/pro',
'strings' => [
```

```
    'modalTitle' => __('Museder RestoreOne PRO Required', 'museder-restoreone'),
    'modalSubtitle' => __('This feature requires Museder RestoreOne PRO to activate.', 'museder-restoreone'),
    'close' => __('Close', 'museder-restoreone'),
    'upgrade' => __('Upgrade to PRO', 'museder-restoreone'),
    /* translators: %s: Name of the locked feature. */
    'featureLocked' => __('Feature "%s" is available in Museder RestoreOne PRO.', 'museder-restoreone'),
    ]]);
```

↳ Detected name: BackupLitePro

```
includes/class-ui.php:116 wp_localize_script('backup-lite-admin-ui', 'BackupLiteAdmin', ['theme' =>
isset($settings['ui_theme']) ? $settings['ui_theme'] : 'auto', 'features' => ['restoreV2' => !empty($settings['feature_
restore_center_v2']), 'animations' => !empty($settings['feature_ui_animation']), 'extendedLog' =>
!empty($settings['feature_extended_log'])]);
```

↳ Detected name: BackupLiteAdmin

```
includes/class-ui.php:129 wp_localize_script('backup-lite-admin-ui', 'BackupLiteAdmin', ['theme' =>
isset($settings['ui_theme']) ? $settings['ui_theme'] : 'auto']);
```

↳ Detected name: BackupLiteAdmin

```
includes/class-ui.php:164 wp_localize_script('backup-lite-admin', 'BackupLite', ['ajaxUrl' => admin_url('admin-
ajax.php'), 'nonce' => wp_create_nonce(self::NONCE), 'nonceV2' => $nonce_v2, 'restUrlV2' => $rest_url_v2, 'page'
=> $current_page, 'activeJob' => $active_job, 'jobPollingInterval' => 3, 'confirmRestore' => __('Restoring will
overwrite your current site files and database. Continue?', 'museder-restoreone'), 'strings' => [
```

```
    'runningTitle' => __('Processing...', 'museder-restoreone'),
    'runningMessage' => __('Please wait while we complete your request.', 'museder-restoreone'),
    'errorTitle' => __('Something went wrong', 'museder-restoreone'),
    'errorGeneric' => __('An unexpected error occurred. Check logs for details.', 'museder-restoreone'),
    'noFileSelected' => __('Please select a backup file to restore.', 'museder-restoreone'),
    'noConfirm' => __('Please confirm you understand the restore impact.', 'museder-restoreone'),
    'successTitle' => __('Completed', 'museder-restoreone'),
    'successBackup' => __('Site backup completed successfully.', 'museder-restoreone'),
    'successRestore' => __('Site restore completed successfully.', 'museder-restoreone'),
    'downloadLabel' => __('Download backup', 'museder-restoreone'),
    'downloadUnavailable' => __('Download link is not available. Please download from the backup library.',
'museder-restoreone'),
    'downloadExpired' => __('Your download link has expired. Please download from the backup library.', 'museder-
```



```

restoreone'),
'noLogs' => __('No log entries yet.', 'museder-restoreone'),
'uploadStarting' => __('Preparing upload...', 'museder-restoreone'),
/* translators: 1: Current chunk number, 2: Total number of chunks. */
'uploadChunk' => __('Uploading chunk %1$s of %2$s...', 'museder-restoreone'),
'merging' => __('Merging uploaded chunks...', 'museder-restoreone'),
'restoring' => __('Restoring site...', 'museder-restoreone'),
/* translators: %s: Number of retries left. */
'retryNotice' => __('Retrying chunk upload... (%1$s attempts left)', 'museder-restoreone'),
'uploadAborted' => __('Upload aborted.', 'museder-restoreone'),
/* translators: %s: Transfer speed in megabytes per second. */
'speed' => __('Speed: %s MB/s', 'museder-restoreone'),
/* translators: %s: Estimated time remaining in seconds. */
'eta' => __('ETA: %s seconds', 'museder-restoreone'),
/* translators: %s: Maximum allowed file size. */
'fileTooLarge' => __('File exceeds maximum allowed size (%s).', 'museder-restoreone'),
/* translators: %s: Number of missing upload chunks. */
'missingChunks' => __('Missing chunks detected: %s', 'museder-restoreone'),
'resumeUpload' => __('Re-uploading missing chunks...', 'museder-restoreone'),
'prepareFailed' => __('Unable to start upload session.', 'museder-restoreone'),
/* translators: %s: SHA1 hash calculated on the server. */
'serverSha1' => __('Server SHA1: %s', 'museder-restoreone'),
/* translators: %s: SHA1 hash calculated on the client. */
'clientSha1' => __('Client SHA1: %s', 'museder-restoreone'),
'sha1Mismatch' => __('Warning: SHA1 mismatch detected!', 'museder-restoreone'),
'errorSha1Mismatch' => __('Uploaded archive failed integrity check. Please re-upload the backup.', 'museder-restoreone'),
'errorZipVerificationFailed' => __('Merged archive could not be validated. Check restore.log for details.', 'museder-restoreone'),
'errorZipOpenFailed' => __('Unable to extract backup archive. See restore.log and debug.log for details.', 'museder-restoreone'),
/* translators: %s: List of allowed file extensions. */
'invalidExtension' => __('Unsupported file extension. Allowed: %s', 'museder-restoreone'),
'confirmDelete' => __('Are you sure you want to delete this backup? This action cannot be undone.', 'museder-restoreone'),
'viewLog' => __('View', 'museder-restoreone'),
'downloadLog' => __('Download', 'museder-restoreone'),
'deleteLog' => __('Delete', 'museder-restoreone'),
'logTruncated' => __('Showing last 200KB (truncated).', 'museder-restoreone'),
'logsRefreshed' => __('Logs refreshed.', 'museder-restoreone'),
'logDeleted' => __('Log deleted.', 'museder-restoreone'),
'settingsSaved' => __('Settings saved successfully.', 'museder-restoreone'),
'confirmDeleteLog' => __('Delete this log file?', 'museder-restoreone'),
'testEmailSuccess' => __('Test email sent successfully.', 'museder-restoreone'),
'copySuccess' => __('Copied to clipboard', 'museder-restoreone'),
'noSchedules' => __('No schedules configured yet.', 'museder-restoreone'),
'scheduleResultSuccess' => __('Success', 'museder-restoreone'),
'scheduleResultFailed' => __('Failed', 'museder-restoreone'),
'scheduleResultPending' => __('Pending', 'museder-restoreone'),
'scheduleEnabled' => __('Enabled', 'museder-restoreone'),
'scheduleDisabled' => __('Disabled', 'museder-restoreone'),
'restoreInProgress' => __('Restore in Progress', 'museder-restoreone'),
'restoreCompleted' => __('Restore Completed', 'museder-restoreone'),
'awaitingRestore' => __('Awaiting restore.', 'museder-restoreone'),
'stepUploadIdle' => __('Choose a backup and run Step 1.', 'museder-restoreone'),
'stepUploadProcessing' => __('Analyzing backup...', 'museder-restoreone'),
'stepUploadDone' => __('Analysis complete. Continue to Step 2.', 'museder-restoreone'),
'stepUploadError' => __('Analysis failed. Try again.', 'museder-restoreone'),

```

```

'stepReviewLocked' => __('Complete Step 1 first to unlock these options.', 'museder-restoreone'),
'stepReviewReady' => __('Options unlocked. Adjust restore behavior.', 'museder-restoreone'),
'stepReviewDone' => __('Options saved. Continue to Step 3.', 'museder-restoreone'),
'stepExecuteLocked' => __('Complete Steps 1 & 2 before starting the restore.', 'museder-restoreone'),
'stepExecuteReady' => __('Ready to start restore.', 'museder-restoreone'),
'stepExecuteProcessing' => __('Restore running...', 'museder-restoreone'),
'stepExecuteDone' => __('Restore finished. Review your site.', 'museder-restoreone'),
'restoreFinalizing' => __('Finalizing restore...', 'museder-restoreone'),
'restoreFinalizingMessage' => __('Completing final steps...', 'museder-restoreone'),
'restoreFailed' => __('Restore Failed', 'museder-restoreone'),
'restoreOverlayMessage' => __('Museder RestoreOne has finished restoring your site.', 'museder-restoreone'),
'restoreOverlayConfirm' => __('Got it', 'museder-restoreone'),
'selectAtLeastOneBackup' => __('Please select at least one backup.', 'museder-restoreone'),
/* translators: %s: Number of backups selected. */
'downloadingBackups' => __('Downloading %s backup(s)...', 'museder-restoreone'),
/* translators: %s: Number of backups deleted. */
'deletedBackups' => __('Deleted %s backup(s).', 'museder-restoreone'),
/* translators: %s: Number of backups that could not be deleted. */
'failedDeleteBackups' => __('Failed to delete %s backup(s). Check logs.', 'museder-restoreone'),
'manualJobStarted' => __('Backup job started manually.', 'museder-restoreone'),
'provideScheduleTitle' => __('Please provide a schedule title.', 'museder-restoreone'),
'scheduleDeleted' => __('Schedule deleted.', 'museder-restoreone'),
'unableDeleteSchedule' => __('Unable to delete schedule.', 'museder-restoreone'),
'licenseError' => __('An error occurred while verifying the license.', 'museder-restoreone'),
'scheduleActionStart' => __('Start Now', 'museder-restoreone'),
'scheduleActionEdit' => __('Edit', 'museder-restoreone'),
'scheduleActionDelete' => __('Delete', 'museder-restoreone'),
'scheduleActionsAria' => __('Schedule actions', 'museder-restoreone'),
'confirmDeleteSelected' => __('Are you sure you want to delete the selected backups? This action cannot be
undone.', 'museder-restoreone'),
'confirmDeleteSchedule' => __('Delete this schedule?', 'museder-restoreone'),
'messageReady' => __('Backup ready for restore.', 'museder-restoreone'),
'confirmOverwriteData' => __('This will overwrite your site data. Continue?', 'museder-restoreone'),
'jobPreparing' => __('Preparing backup...', 'museder-restoreone'),
'jobQueued' => __('Waiting for server resources...', 'museder-restoreone'),
'jobProcessing' => __('Processing files...', 'museder-restoreone'),
'jobFinalizing' => __('Finalising backup archive...', 'museder-restoreone'),
'jobComplete' => __('Backup completed successfully.', 'museder-restoreone'),
'jobFailed' => __('Backup failed. Check logs for details.', 'museder-restoreone'),
'jobResuming' => __('Resuming previous backup job...', 'museder-restoreone'),
'jobNudging' => __('Continuing backup in the background...', 'museder-restoreone'),
'jobCancelled' => __('Backup cancelled.', 'museder-restoreone'),
'jobButtonBusy' => __('Backup in progress...', 'museder-restoreone'),
'jobButtonIdle' => __('Backup Site', 'museder-restoreone'),
'jobCancelConfirm' => __('Cancel the running backup job?', 'museder-restoreone'),
'jobCancelSuccess' => __('Backup job cancelled.', 'museder-restoreone'),
'jobCancelFailed' => __('Unable to cancel backup job. Please try again.', 'museder-restoreone'),
'restoreCancelConfirm' => __('Cancel the current restore process?', 'museder-restoreone'),
'restoreCancelSuccess' => __('Restore process cancelled.', 'museder-restoreone'),
'restoreCancelFailed' => __('Unable to cancel restore process. Please try again.', 'museder-restoreone'),
'chunkPreparing' => __('Preparing upload...', 'museder-restoreone'),
/* translators: 1: Current chunk number, 2: Total chunks, 3: Progress percentage. */
'chunkUploading' => __('Uploading %1$s of %2$s (%3$s%)...', 'museder-restoreone'),
'chunkMerging' => __('Merging uploaded chunks...', 'museder-restoreone'),
], 'chunk' => ['prepareAction' => 'backup_lite_prepare_upload', 'chunkSize' => 2 * 1024 * 1024, 'maxFileSize' =>
4294967296, 'uploadAction' => 'backup_lite_upload_chunk', 'finalizeAction' => 'backup_lite_finalize_upload',
'abortAction' => 'backup_lite_abort_upload', 'allowedExt' => ['zip', 'wpress']]);
# ↳ Detected name: BackupLite

```

```

includes/class-ui.php:304 wp_localize_script('backup-lite-chunk-upload-v2', 'BackupLiteV2', ['restUrl' =>
esc_url_raw($rest_url_v2), 'nonce' => wp_create_nonce('wp_rest')]);
# ↳ Detected name: BackupLiteV2
includes/class-backup-lite-s3-service.php:496 apply_filters('museder_restoreone_s3_upload_timeout', 600);
includes/class-cloud-service.php:89 update_option(self::OPTION_KEY, $settings, false);
# ↳ Detected name: museder_cloud_settings
includes/class-cloud-service.php:20 class Museder_RestoreOne_Cloud_Service
museder-restoreone.php:249 wp_localize_script('backup-lite-dashboard', 'BackupLiteDashboard', ['chart' =>
['success' => $chart_success, 'failed' => $chart_failed], 'nextRunTimestamp' => $next_run, 'strings' => ['dueNow'
=> __('Due now', 'museder-restoreone'), 'noData' => __('No activity recorded in the last 7 days.', 'museder-
restoreone'), 'successLabel' => __('Success', 'museder-restoreone'), 'failedLabel' => __('Failed', 'museder-
restoreone')]);
# ↳ Detected name: BackupLiteDashboard
museder-restoreone.php:319 wp_localize_script('backup-lite-restore', 'BackupLiteRestore', ['restURL' =>
esc_url_raw(rest_url('backup-lite/v2/')), 'nonce' => wp_create_nonce('wp_rest'), 'ajaxUrl' => admin_url('admin-
ajax.php'), 'ajaxNonce' => wp_create_nonce(Backup_Lite_UI::NONCE), 'siteURL' => home_url(), 'uploads' =>
trailingslashit(backup_lite_get_storage_root()['path']), 'cap' => current_user_can('manage_options'), 'backups' =>
$museder_restoreone_backups, 'summary' => $museder_restoreone_summary, 'progress' =>
$museder_restoreone_progress, 'history' => $museder_restoreone_history, 'job' => $museder_restoreone_active_job
? Backup_Lite_Restore_Jobs::prepare_job_response($museder_restoreone_active_job) : null, 'labels' =>
['noBackups' => __('No backups available.', 'museder-restoreone'), 'noValidation' => __('Validation results will
appear here once the job is prepared.', 'museder-restoreone'), 'selectFile' => __('Please choose a backup file first.',
'museder-restoreone'), 'noDryRun' => __('Dry-run results will appear here once available.', 'museder-restoreone')]);
# ↳ Detected name: BackupLiteRestore
museder-restoreone.php:447 wp_localize_script('backup-lite-reports', 'BackupLiteReports', ['restUrl' =>
esc_url_raw(rest_url('backup-lite/v2/')), 'nonce' => wp_create_nonce('wp_rest'), 'isPro' => $is_pro]);
# ↳ Detected name: BackupLiteReports

```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-cloud-service.php:89 update_option(self::OPTION_KEY, $settings, false);
```

Unsafe SQL calls

When making database calls, it's highly important to protect your code from SQL injection vulnerabilities. You need to update your code to use wpdb calls and prepare() with your queries to protect them.

Please review the following:

- <https://developer.wordpress.org/reference/classes/wpdb/#protect-queries-against-sql-injection-attacks>
- https://codex.wordpress.org/Data_Validation#Database
- <https://make.wordpress.org/core/2012/12/12/php-warning-missing-argument-2-for-wpdb-prepare/>
- <https://ottopress.com/2013/better-know-a-vulnerability-sql-injection/>

Example(s) from your plugin:

```
includes/class-restore.php:580 $safe = backup_lite_validate_servmask_table_name( $table );
includes/class-restore.php:589 $wpdb->query(
// phpcs:ignore PluginCheck.Security.DirectDB.UnescapedDBParameter -- Identifier cannot be passed as a
prepared value; validated via backup_lite_validate_servmask_table_name() (live SERVMASK whitelist).
// phpcs:ignore WordPress.DB.PreparedSQL.InterpolatedNotPrepared -- Identifier cannot be prepared; validated
via backup_lite_validate_servmask_table_name() (live SERVMASK whitelist).
"DROP TABLE IF EXISTS `{$safe}`"
);
# You cannot add calls like "backup_lite_validate_servmask_table_name($table)" directly to the SQL query.
# Using wpdb::prepare($query, $args) you will need to include placeholders for each variable within the query and
include the variables in the second parameter.
# The SQL query needs to be included in a wpdb::prepare($query, $args) function.

includes/class-restore.php:1530 $prepared = trim( $query );
includes/class-restore.php:1539 $result = $wpdb->query( $prepared ); // phpcs:ignore
WordPress.DB.PreparedSQL.NotPrepared -- $prepared is a complete SQL script from backup file, cannot use
prepare() for multi-statement scripts
-----> $wpdb->query($prepared)
# The SQL query needs to be included in a wpdb::prepare($query, $args) function.
# Remember that you will need to include placeholders for each variable within the query and include their calls in
the second parameter of wpdb::prepare().
```

 **Continue with the review process.**

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

 **Complete your checklist.**

✓ **I fixed all the issues** in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. **I am familiar with tools** such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.

✓ **I tested my updated plugin on a clean WordPress installation** with [WP_DEBUG set to true](#).

 **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

- ✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.
- ✓ I went to "[Add your plugin](#)" and uploaded the updated version. *I can continue updating the code there throughout the review process — the team will always check the latest version.*
- ✓ I replied to **this** email. I was concise and shared any clarifications or important context that the team needed to know.
I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

i To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, **we ask you to thoroughly check all shared issues and fix them before sending the code back to us.** *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T3 21Dec25/3.7.1